STUDENT FEE PORTAL

PROJECT DOCUMENTATION

A Web-Based Student Fee Management Application

Prepared by: *Aisha Kabiru*

Department: *Frontend development /Intermediate*

Session: *2025/2026*

Project Links

Live Website: https://scintillating-lamington-f05ef5.netlify.app

GitHub Repository: https://github.com/17835/student-fee-portal

2026

1. INTRODUCTION

The Student Fee Portal is a web-based application developed to help students view and manage information relating to their school fees. The application provides a central interface where students can review fee information, navigate payment-related pages, and access supporting features.

The project demonstrates the use of modern web development technologies to create a responsive, organized, and user-friendly application. It was developed as an academic project to apply Software Engineering concepts, particularly frontend development, component-based design, and application navigation.

The Student Fee Portal is an academic prototype and is not an official university payment system.

2. PROBLEM STATEMENT

Students need a convenient way to review their school fee obligations and understand their payment status. When fee information and payment-related records are difficult to access or are presented across different channels, students may find it challenging to monitor their obligations.

The Student Fee Portal addresses this challenge by presenting fee-related information through a single interface. It aims to make the process of reviewing fees and navigating payment-related information clearer and more organized.

3. AIM AND OBJECTIVES

3.1 Aim

The aim of this project is to develop a responsive web-based Student Fee Portal that enables students to view fee information and navigate payment-related tasks through a user-friendly interface.

3.2 Objectives

The objectives of the project are to:

1. Develop a dashboard that presents important student fee information.
2. Display individual fee categories and their associated amounts and statuses.
3. Provide an interface for demonstrating the fee payment process.
4. Organize receipt and transaction-related information where supported.
5. Provide help and support information.
6. Create a consistent navigation structure across the application.
7. Apply responsive design principles to improve usability across different screen sizes.
8. Practice the use of modern frontend development tools and technologies.

4. SCOPE OF THE PROJECT

The project focuses on the design and development of a student-facing fee management interface. Its scope includes displaying fee information, navigating between application pages, and providing payment-related interfaces.

The application includes pages such as Dashboard, Fee Breakdown, Payments, Receipts, Help & Support, and Settings, subject to the features available in the deployed version.

The project is intended for academic demonstration. Real payment processing, secure banking integration, and official university financial records are outside its scope unless separately implemented and verified.

5. MAIN FEATURES OF THE APPLICATION

5.1 Dashboard

The Dashboard serves as the main page of the application. It provides an overview of student fee information and helps users understand their fee status without having to navigate through every section individually.

5.2 Fee Breakdown

The Fee Breakdown page presents the individual fee categories and their associated amounts and payment statuses. This helps users review the different components of their school fees.

5.3 Payments

The Payments page provides the interface for the fee payment workflow. It demonstrates how a student can interact with payment-related forms and information within the application. Payment interactions should be treated as demonstrations unless a real payment provider has been integrated and verified.

5.4 Receipts

The Receipts page provides access to receipt-related information. Where supported by the application, users can generate or download receipts in PDF format for record-keeping purposes.

5.5 Help and Support

The Help and Support page provides a dedicated area for assistance-related information and guidance within the application.

5.6 Settings

The Settings page groups available application preferences and configuration options, depending on the functionality implemented in the deployed version.

5.7 Responsive User Interface

The application is designed to provide a clear and organized interface across common screen sizes. Its layout and navigation are intended to make the application easier to use on desktop and smaller screens.

6. TOOLS AND TECHNOLOGIES

The project uses web development technologies and supporting tools to build and run the application.

6.1 React

React is a JavaScript library used to build user interfaces from reusable components. It supports the organization of the application into separate pages and interface elements.

6.2 JavaScript

JavaScript is used to implement application behavior, user interactions, and the logic that connects interface components.

6.3 HTML5

HTML5 provides the structure and semantic elements used to organize the content displayed in the web application.

6.4 CSS

CSS is used to style the application, control layouts, and improve the visual presentation of the interface.

6.5 Vite

Vite is a frontend development and build tool used to run the development server and prepare the application for deployment, where configured in the project.

6.6 Axios

Axios is a JavaScript library used to make HTTP requests to an API. In this project, it can be used to communicate with the configured data service.

6.7 JSON Server

JSON Server can provide a mock REST API for development and testing. If the application uses a local JSON Server instance, that service must be separately hosted and configured before it can serve data to the public deployment.

6.8 Additional Libraries and Browser Storage

Additional tools such as Chart.js, html2pdf.js, and browser LocalStorage may support charts, PDF generation, and data persistence where they are implemented in the application.

Only technologies confirmed in the final project source code should be presented as implemented features.

7. APPLICATION WORKFLOW

The Student Fee Portal follows a simple workflow designed to help users navigate the application's main features.

1. **Access the application:** The user opens the deployed website or runs the application in a local development environment.
2. **View the dashboard:** The user begins at the Dashboard to review the fee information displayed.
3. **Review fee details:** The user navigates to the Fee Breakdown page to examine individual fee categories and their associated statuses.
4. **Explore payment features:** The user opens the Payments page to interact with the available payment interface.

5. **Access receipts:** The user visits the Receipts page to review receipt-related information where available.
6. **Use supporting pages:** The user can access Help & Support and Settings through the application's navigation.

The exact data and interactions available depend on the application's current configuration and deployment.

8. TESTING AND EVALUATION

Testing is important for checking that the application works as intended and provides a usable experience.

The following checks can be performed on the Student Fee Portal:

1. **Page loading:** Confirm that the deployed website opens successfully.
2. **Navigation:** Check that the menu links open the correct pages.
3. **Fee information:** Verify that the fee categories, amounts, and displayed statuses are presented correctly.
4. **Form validation:** Test required fields and validation messages in the payment interface.
5. **Receipt functionality:** Check that receipt information and PDF generation work if implemented.
6. **Responsive design:** Review the layout at desktop and mobile screen widths.
7. **Error handling:** Check how the application responds when data cannot be loaded or an operation fails.

Testing results should be recorded after the checks have actually been performed. Features that have not been tested should not be described as successfully tested.

9. LIMITATIONS OF THE PROJECT

Although the Student Fee Portal demonstrates a centralized interface for student fee information, it has limitations.

First, it is an academic prototype rather than an official university financial system. Its payment interface should not be assumed to process real payments.

Second, any mock API or local development service may not be accessible through the public deployment unless it has been separately hosted and configured.

Third, the reliability of stored information depends on the application's data source and implementation. A demonstration environment may not provide the security, availability, and data integrity required by a real institution.

Finally, additional security controls, accessibility improvements, and comprehensive automated testing may be needed before the application could be considered for production use.

10. FUTURE IMPROVEMENTS

The following improvements could strengthen the application:

1. Integrate an approved payment provider in a secure test environment before any production use.
2. Implement secure authentication and appropriate access controls.
3. Connect the application to a reliable hosted backend and database.
4. Improve error messages and guidance when a service is unavailable.
5. Expand automated testing and accessibility checks.
6. Improve notification features for upcoming fee deadlines.
7. Collect feedback from students and use it to improve the interface.

11. CONCLUSION

The Student Fee Portal demonstrates the development of a web-based interface for organizing student fee information and payment-related features. It brings several related pages into one application and provides an opportunity to apply frontend development, reusable components, navigation, and responsive design principles.

The project provides a foundation for further development. With additional testing, secure backend services, appropriate data protection, and verified payment integration, the concept could be extended into a more comprehensive fee management solution.

12. PROJECT LINKS

Live Website:

https://scintillating-lamington-f05ef5.netlify.app

GitHub Repository:

https://github.com/17835/student-fee-portal